



DWN - Unid II

Exercício 1:

Uma aplicação inicialmente armazena produtos em uma `List<Produto>` criada durante a inicialização do processo. Posteriormente, passa a utilizar SQLite com Entity Framework Core e um arquivo `dados.db`.

Qual mudança é mais relevante do ponto de vista da persistência?

A)

Os registros deixam de depender exclusivamente da duração do processo e podem ser recuperados do banco em execuções posteriores.

B)

Os objetos deixam de existir em memória e passam a ser manipulados diretamente no arquivo pelo código C#.

C)

O SQLite preserva os mesmos objetos C# em memória depois que o processo é encerrado.

D)

O banco elimina a necessidade de representar registros por objetos durante a execução da aplicação.

E)

Qualquer alteração feita em uma entidade é gravada imediatamente no SQLite, independentemente de `SaveChanges()`.

Exercício 2:

Considere a configuração:

```
builder.Services.AddDbContextPool<AppDbContext>(options =>  
    options.UseSqlite(  
        "Data Source=dados.db;Pooling=True"));
```

Qual alternativa diferencia corretamente os dois mecanismos de pooling presentes nesse cenário?

A)

Pooling=True reutiliza instâncias de DbContext, enquanto AddDbContextPool() reutiliza conexões com o SQLite.

B)

Ambos configuram o mesmo pool, e utilizá-los juntos apenas duplica sua capacidade.

C)

AddDbContextPool() compartilha entidades consultadas entre requisições, enquanto Pooling=True mantém resultados SQL em cache.

D)

O parâmetro da conexão atua no pooling de conexões do provedor, enquanto AddDbContextPool() permite reutilizar instâncias de DbContext depois da reinicialização de seu estado.

E)

Pooling=True preserva transações concluídas, enquanto AddDbContextPool() mantém migrations em memória.

Exercício 3:

Uma entidade utiliza o seguinte mapeamento:

```
public int Id { get; set; }
```

```
public string CategoriasTexto { get; set; } = "";
```

```
[NotMapped]
```

```
public string[] Categorias  
{
```

```
get => CategoriasTexto.Split(
    '|',
    StringSplitOptions.RemoveEmptyEntries |
    StringSplitOptions.TrimEntries);

set => CategoriasTexto = string.Join('|', value);
}
```

Qual interpretação está correta?

A)

Categorias e CategoriasTexto são persistidas como duas colunas, mantidas automaticamente em sincronismo pelo EF Core.

B)

CategoriasTexto é persistida, enquanto Categorias oferece uma representação conveniente em memória sem possuir coluna própria.

C)

Categorias representa uma relação persistente com outra tabela, enquanto CategoriasTexto existe apenas para a interface.

D)

[NotMapped] faz com que toda a entidade deixe de participar do rastreamento de alterações.

E)

[NotMapped] permite que a propriedade seja gravada em inserções, mas impede sua leitura em consultas.

Exercício 4:

Uma aplicação que utiliza SQLite adota concorrência otimista com um token gerenciado pela própria aplicação. A entidade possui:

```
public Guid Version { get; set; } = Guid.NewGuid();
```

O mapeamento contém:

```
builder.Property(x => x.Version)
    .IsConcurrencyToken();
```

A aplicação atribui um novo Guid a Version antes de cada SaveChangesAsync() e

usa como valor original o token recebido pelo cliente. Dois usuários, A e B, carregam a mesma versão de um registro. A salva primeiro, fazendo o token mudar. Em seguida, B tenta salvar com o token original recebido ao abrir o registro.

O que pode ocorrer na tentativa de B?

A)

A alteração de B necessariamente substitui a de A, porque a última gravação sempre prevalece.

B)

A alteração de A é desfeita automaticamente quando B envia um token de versão anterior.

C)

O EF Core bloqueia o registro desde a consulta inicial, impedindo que A e B o editem simultaneamente.

D)

O EF Core identifica as propriedades modificadas por cada usuário e combina automaticamente as duas alterações.

E)

A atualização de B pode não afetar nenhuma linha e `SaveChangesAsync()` pode lançar `DbUpdateConcurrencyException`.

Exercício 5:

Considere a consulta:

```
var consulta = dbContext.Produtos
    .AsNoTracking()
    .Where(p => p.Ativo);
```

```
if (!string.IsNullOrEmpty(categoria))
{
    consulta = consulta.Where(
        p => p.Categoria == categoria);
}
```

```
var produtos = await consulta
    .OrderBy(p => p.Nome)
    .Select(p => new ProdutoResponse(
        p.Id,
        p.Nome,
        p.Categoria,
        p.Preco))
    .ToListAsync();
```

Qual análise está correta?

A)

AsNoTracking() impede futuras atualizações da tabela, e os filtros posteriores são aplicados somente depois que os registros são carregados.

B)

Select() exige primeiro materializar entidades completas e somente depois descarta as propriedades não utilizadas.

C)

A consulta continua sendo composta até a materialização; AsNoTracking() evita o rastreamento das entidades nesse cenário de leitura; e Select() define a projeção do resultado.

D)

Em razão da execução diferida do LINQ, ToListAsync() ocorre antes de Where() e OrderBy().

E)

AsNoTracking() cria um cache local dos registros e Select() transforma a operação em uma transação somente leitura.

Exercício 6:

Uma entidade de persistência contém propriedades internas utilizadas pelo Entity Framework Core, inclusive um token de concorrência. O endpoint responde com:

```
public sealed record ProdutoResponse(
    int Id,
    string Nome,
```

```
string Categoria,  
decimal Preco,  
bool Disponivel);
```

O objeto retornado é criado por projeção a partir da entidade. Qual é a principal vantagem dessa decisão?

A)

O DTO impede que propriedades omitidas sejam consultadas pelo EF Core em qualquer outra operação.

B)

A criação do DTO elimina a entidade do modelo de persistência depois que a consulta é executada.

C)

O DTO cria automaticamente novas versões da API sempre que a entidade sofre uma alteração.

D)

O contrato HTTP fica separado do modelo de persistência, permitindo definir explicitamente quais dados são expostos ao cliente.

E)

A utilização de DTO torna desnecessário documentar os schemas e códigos de resposta da API.

Exercício 7:

Uma API recebe os parâmetros:

```
public class ProdutoQuery  
{  
    private const int MaxPageSize = 20;  
  
    public int Page { get; set; } = 1;  
  
    private int _pageSize = 6;
```

```
public int PageSize
{
    get => _pageSize;
    set => _pageSize =
        Math.Clamp(value, 1, MaxPageSize);
}
```

A paginação é aplicada assim:

```
var page = Math.Max(query.Page, 1);
```

```
var totalItems = await consulta.CountAsync();
```

```
var items = await consulta
    .Skip((page - 1) * query.PageSize)
    .Take(query.PageSize)
    .ToListAsync();
```

Há 47 registros. O cliente envia:

?page=3&pageSize=50

Qual resultado decorre dessa lógica?

A)

A página efetiva é 3 e o tamanho permanece 50, permitindo que os 47 registros sejam devolvidos.

B)

A página efetiva é 3, o tamanho é limitado a 20, são ignorados 40 registros e podem ser retornados no máximo 7 itens.

C)

Como pageSize ultrapassa o máximo, page e pageSize retornam aos valores padrão.

D)

O model binding responde necessariamente com 400 Bad Request, pois Math.Clamp() rejeita valores superiores ao máximo.

E)

Os valores são invertidos, fazendo a terceira página conter 20 páginas com 3 registros cada.

Exercício 8:

Um endpoint retorna uma resposta estruturada quando não encontra determinado produto:

```
return produto is null
```

```
    ? Results.Problem(  
        title: "Produto não encontrado.",  
        detail: $"Não existe produto com id {id}.",  
        statusCode: StatusCodes.Status404NotFound)  
    : Results.Ok(produto);
```

A aplicação também utiliza os serviços de ProblemDetails.

Qual é a vantagem de representar a falha dessa maneira?

A)

Transferir ao cliente a responsabilidade de definir qual código HTTP corresponde ao problema informado.

B)

Converter a resposta 404 em 200, permitindo que o cliente determine o erro somente pelo conteúdo recebido.

C)

Utilizar uma única resposta para qualquer falha, sem necessidade de distinguir erros como 400, 404 ou 409.

D)

Fazer com que o roteamento tente localizar automaticamente outro produto quando o identificador não existir.

E)

Manter o código HTTP 404 e fornecer ao cliente uma representação estruturada com informações sobre o problema.

Exercício 9:

Uma equipe utiliza OpenAPI, Swagger UI e NSwag em uma API ASP.NET Core. Qual alternativa descreve adequadamente a relação entre essas tecnologias?

A)

OpenAPI descreve formalmente o contrato da API; Swagger UI pode apresentar esse contrato de forma interativa; e NSwag pode consumir um documento OpenAPI para gerar código cliente.

B)

Swagger UI e NSwag são equivalentes porque ambos precisam apresentar a mesma interface gráfica ao usuário.

C)

OpenAPI é uma interface gráfica; Swagger UI gera exclusivamente clientes TypeScript; e NSwag define a especificação HTTP utilizada pela API.

D)

Swagger define o contrato durante o desenvolvimento, enquanto OpenAPI existe apenas para geração de clientes em produção.

E)

Depois que o NSwag gera o cliente, a Swagger UI passa a inferir os endpoints diretamente pelo código TypeScript.

Exercício 10:

Um projeto ASP.NET Core direcionado ao .NET 10 referencia os pacotes Microsoft.Extensions.ApiDescription.Server e NSwag.MSBuild. O arquivo .csproj contém:

```
<PropertyGroup>
  <OpenApiGenerateDocuments>true</OpenApiGenerateDocuments>
  <OpenApiDocumentsDirectory>
    $(MSBuildProjectDirectory)/obj/openapi
  </OpenApiDocumentsDirectory>
  <OpenApiGenerateDocumentsOptions>
    --file-name v1
  </OpenApiGenerateDocumentsOptions>
</PropertyGroup>
```

```
<Target Name="GenerateTypeScriptClient"
  AfterTargets="GenerateOpenApiDocuments">
  <Error
    Condition="!Exists('$(OpenApiDocumentsDirectory)/v1.json')"
    Text="Documento OpenAPI não encontrado." />

  <Exec WorkingDirectory="$(MSBuildProjectDirectory)"
    Command="$(NSwagExe_Net100) run nswag.json" />
</Target>
```

O arquivo nswag.json usa como entrada:

obj/openapi/v1.json

e gera:

Clients/TypeScript/api-client.ts

Considerando que os dois pacotes estão instalados em versões compatíveis com o projeto, qual sequência é coerente com essa configuração?

A)

O NSwag cria v1.json; depois a Swagger UI transforma o arquivo em TypeScript.

B)

A Swagger UI precisa permanecer aberta durante o build para fornecer o documento ao NSwag.

C)

O build gera primeiro v1.json no diretório configurado; depois o target executa o NSwag para gerar o cliente TypeScript a partir desse documento.

D)

O cliente TypeScript é criado antes do documento e passa a ser usado para reconstruir v1.json.

E)

A geração do OpenAPI e a do cliente são independentes, portanto a existência de v1.json não interfere no target.